

Architecture Selection and Justification

Remote Patient Vitals Alert and Monitoring App

I chose Microservices Architecture for the Remote Patient Vitals Alert and Monitoring App. The system receives SpO2, heart rate and blood pressure readings from post-operative patients, detects threshold breaches, and escalates alerts through an on-call caregiver matrix.

Architectural style comparison

Style	Assessment for this system
Layered	Clear separation and simpler development. A single deployment couples scaling and releases across ingestion, analysis and alerts.
Client-Server	Fits remote apps accessing a central backend. By itself, it does not define independent scaling or isolation of monitoring functions.
Microservices	Separates independently deployable services and permits targeted scaling. Adds network latency, deployment complexity and consistency challenges.

Two reasons for this choice

1. Independent scaling. Ingestion and analysis can be replicated separately as telemetry grows toward 500 concurrent streams. Notification delivery can scale with alert volume without requiring every backend service to scale together.
2. Failure isolation. Separate notification workers, bounded timeouts and persisted pending alerts prevent a slow delivery provider from blocking ingestion and threshold evaluation. The alert service tracks acknowledgements and advances through the caregiver matrix after a configurable timeout.

Security advantage

Authentication issues signed user or device tokens. Each exposed service validates credentials and checks patient-specific permissions, restricting caregivers to assigned patients. TLS protects communication; internal service credentials and restricted data APIs reduce unauthorized access to patient records.

Performance benefit and availability

Dedicated analysis workers and cached, versioned thresholds target detection within 2 seconds of receipt. Independent notification workers target delivery of at least 95% of critical notifications within 5 seconds after detection. Supporting 500 streams and 99.99% monthly gateway availability requires load testing, redundant instances and recovery monitoring. The shared Data Service needs partitioning and replicas to avoid a bottleneck. These are design targets, not measured results.